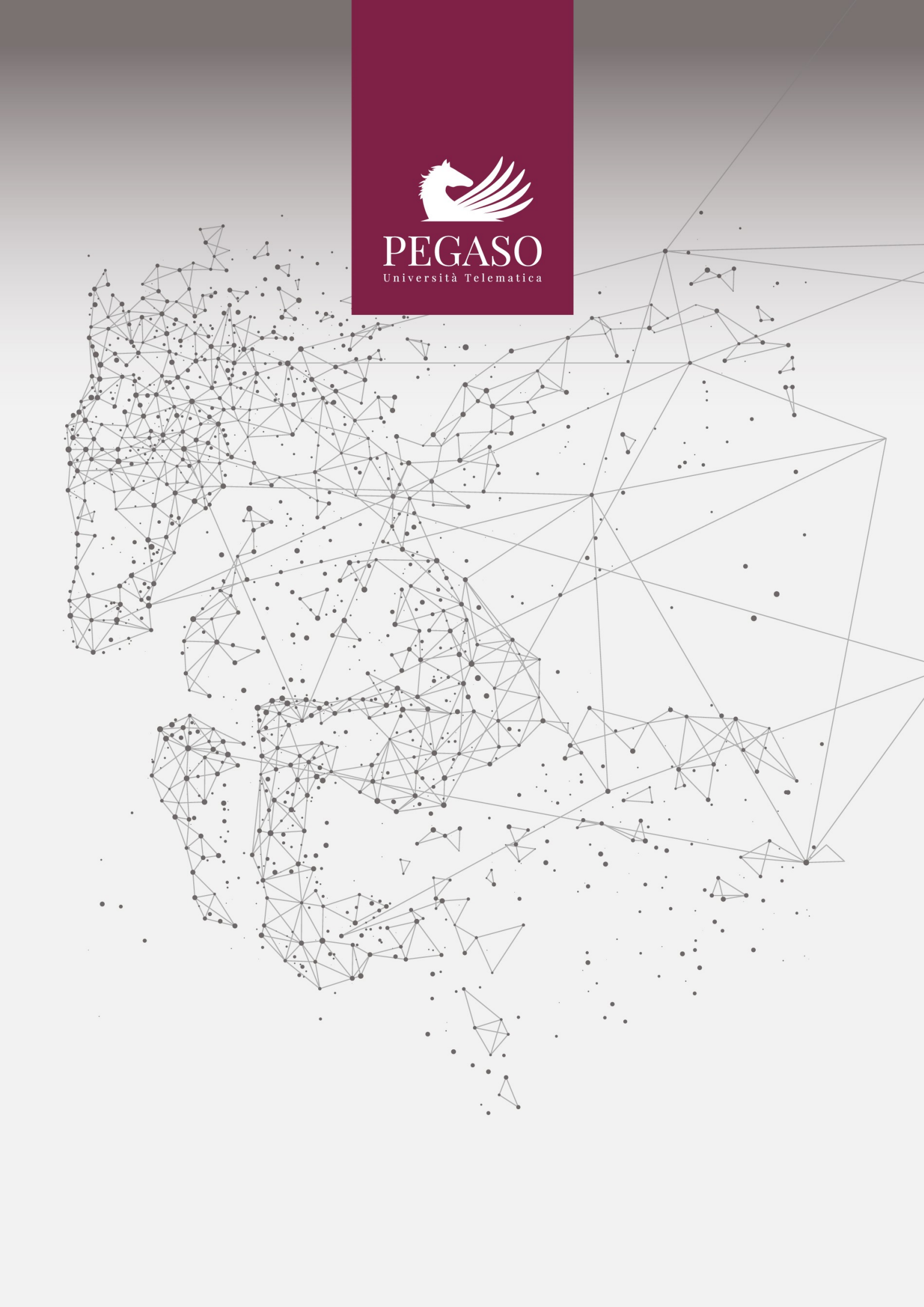

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. AJAX 3

2. FETCH API – PROMISE 6

3. CORS 9

3.1 ACCESS-CONTROL-ALLOW-ORIGIN 9
3.2 PROXY SERVER 10
3.3 JSONP 10

BIBLIOGRAFIA 14

SITOGRAFIA 15

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Ajax

Le chiamate **sincrone** e **asincrone** sono un concetto fondamentale per comprendere il funzionamento di AJAX.

Una chiamata sincrona, o bloccante, è un tipo di chiamata in cui il thread principale di esecuzione del programma viene bloccato fino a quando la chiamata non viene completata. In altre parole, quando si effettua una chiamata sincrona, il programma non può eseguire alcuna altra operazione fino a quando la chiamata non è stata completata. Questo comportamento è spesso indesiderato in applicazioni web, poiché può causare il blocco dell'interfaccia utente e una cattiva esperienza utente.

Al contrario, una chiamata asincrona, o non bloccante, è un tipo di chiamata in cui il thread principale di esecuzione del programma non viene bloccato durante l'esecuzione della chiamata. In altre parole, quando si effettua una chiamata asincrona, il programma può continuare a eseguire altre operazioni, senza dover attendere il completamento della chiamata. Questo comportamento è molto utile in applicazioni web, poiché consente di creare interfacce utente fluide e reattive.

AJAX (Asynchronous JavaScript and XML) è una tecnica di programmazione che consente di eseguire richieste HTTP asincrone utilizzando JavaScript: l'obiettivo di AJAX è quello di creare interfacce utente dinamiche e reattive che non richiedono il ricaricamento completo della pagina web. Quando dunque viene eseguita una richiesta AJAX, il programma continua ad eseguire altre operazioni, senza dover attendere il completamento della richiesta; quando la risposta del server è disponibile, viene gestita in modo asincrono, senza interrompere l'esecuzione del programma.

Per implementare AJAX, è necessario utilizzare JavaScript per eseguire una richiesta HTTP asincrona al server; **XMLHttpRequest** e **Fetch API** sono dei metodi "nativi" perchè direttamente implementati nel browser; tuttavia, ci sono anche altre librerie che consentono di semplificare tali chiamate, come ad es. jQuery, Axios, SuperAgent ecc.

Le due principali API utilizzate per effettuare richieste AJAX nei browser sono **XMLHttpRequest** (**XHR**) e **Fetch API**. XHR è stata la prima API implementata per effettuare richieste AJAX, mentre Fetch API è stata introdotta più di recente con l'obiettivo di semplificare l'API XHR e fornire funzionalità aggiuntive.

La Fetch API è progettata per essere più semplice e flessibile rispetto all'oggetto XMLHttpRequest (XHR) utilizzato tradizionalmente per le richieste http; in particolare, la Fetch API utilizza una sintassi più semplice e intuitiva basata su **Promises** e ci consente di utilizzare i dati della risposta in modo più diretto e conveniente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Una **Promise** rappresenta un'operazione asincrona che potrebbe non essere ancora completata e ci consente di gestire il risultato dell'operazione una volta che è disponibile.

In pratica, le Promises ci permettono di gestire il flusso di esecuzione di una serie di operazioni asincrone in modo più intuitivo e leggibile, senza dover annidare callback all'interno di callback. Possiamo utilizzare le Promises per eseguire operazioni asincrone come le richieste HTTP, le animazioni, le operazioni di elaborazione dati e altre.

Ecco un esempio di come effettuare una richiesta AJAX per ottenere un file JSON utilizzando sia l'API XHR che Fetch API:

```
// XHR
const xhr = new XMLHttpRequest();
xhr.open('GET', 'https://example.com/data.json');
xhr.onload = () => {
  if (xhr.status === 200) {
    const data = JSON.parse(xhr.responseText);
    console.log(data);
  }
};
xhr.send();

// FETCH API
fetch('https://example.com/data.json')
  .then(response => {
    if (response.ok) {
      return response.json();
    }
    throw new Error('Network response was not ok.');
})
.then(data => console.log(data))
.catch(error => console.error(error));
```

jQuery fornisce un metodo chiamato $.ajax() che semplifica la creazione di richieste AJAX personalizzate, offrendo un'interfaccia più semplice rispetto all'utilizzo diretto di XHR o Fetch API.

Ecco un esempio di utilizzo del metodo $.ajax() per effettuare una richiesta GET al server e ricevere una risposta in formato JSON:

```
$.ajax({
  url: 'https://example.com/data.json',
  method: 'GET',
  dataType: 'json',
  success: function(data) {
    console.log(data);
  },
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
  error: function(jqXHR, textStatus, errorThrown) {
    console.error(textStatus, errorThrown);
  }
```

La scelta di jQuery per chiamate AJAX rispetto all'utilizzo di Fetch API risulta comunque una scelta del tutto personale.

È vero che la Fetch API ha molti vantaggi rispetto a jQuery.ajax(), ad esempio:

- è inclusa di default in tutti i moderni browser web;
- supporta i Promise, che semplificano la gestione delle richieste asincrone e consentono di scrivere codice più leggibile e meno propenso ad errori;
- supporta nativamente i dati in formato JSON, senza la necessità di doverli parsare manualmente come avviene con jQuery.ajax();
- consente di inviare richieste con diversi metodi HTTP, non solo GET e POST come in jQuery.ajax();
- consente di inviare richieste con body, che possono contenere diversi tipi di dati, tra cui anche JSON;
- consente di definire custom headers per la richiesta.
- tuttavia, ci sono alcuni casi in cui può essere utile utilizzare jQuery.ajax() invece della Fetch API:
- se si utilizza jQuery in modo globale nell'applicazione e si desidera utilizzare le funzionalità AJAX fornite da jQuery senza dover importare ulteriori librerie;
- se si desidera utilizzare una sintassi più familiare e semplificata per gestire le richieste AJAX, poiché la sintassi di jQuery.ajax() è più breve rispetto a quella della Fetch API;
- se si deve gestire un vecchio browser che non supporta la Fetch API.

In generale, è sempre importante scegliere la tecnologia più adatta alle proprie esigenze e al contesto specifico, valutando i vantaggi e gli svantaggi di ciascuna opzione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Fetch API – Promise

Consideriamo il seguente esempio in cui mostriamo le potenzialità di Fetch API e delle Promise:

```
fetch('https://jsonplaceholder.typicode.com/todos/1')
  .then(response => {
    // Gestione del primo then
    console.log('Primo then:', response);
    return response.json();
  })
  .then(data => {
    // Gestione del secondo then
    console.log('Secondo then:', data);
    return data.userId;
  })
  .then(userId => {
    // Gestione del terzo then
    console.log('Terzo then:', userId);
    // Chiamata AJAX utilizzando l'ID dell'utente
    return fetch(`https://jsonplaceholder.typicode.com/users/${userId}`);
  })
  .then(response => {
    // Gestione del quarto then
    console.log('Quarto then:', response);
    return response.json();
  })
  .then(userData => {
    // Gestione del quinto then
    console.log('Quinto then:', userData);
  })
  .catch(error => {
    // Gestione degli errori
    console.error('Errore:', error);
  });
```

In questo esempio, viene effettuata una chiamata AJAX alla risorsa https://jsonplaceholder.typicode.com/todos/1 utilizzando la Fetch API. Viene poi utilizzato il metodo .then() per gestire la risposta ricevuta dal server.

- Il primo .then() viene utilizzato per verificare che la risposta sia stata ricevuta correttamente e per estrarre i dati JSON dalla risposta utilizzando il metodo .json().
- Il secondo .then() viene utilizzato per gestire i dati JSON ottenuti nella fase precedente e per estrarre l'ID dell'utente dal campo "userId" presente nei dati JSON.
- Il terzo .then() viene utilizzato per gestire l'ID dell'utente estratto nella fase precedente e per effettuare un'altra chiamata AJAX alla risorsa https://jsonplaceholder.typicode.com/users/{userId} utilizzando l'ID dell'utente come parametro.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- Il quarto .then() viene utilizzato per verificare che la risposta alla seconda chiamata AJAX sia stata ricevuta correttamente e per estrarre i dati JSON dalla risposta utilizzando il metodo .json().
- Infine, il quinto .then() viene utilizzato per gestire i dati JSON ottenuti nella fase precedente e per mostrare i dati dell'utente nella console.

Se si verifica un errore in qualsiasi fase del processo, viene utilizzato il metodo .catch() per gestirlo e mostrarlo nella console.

Questo esempio dimostra come le Promise in JavaScript consentano di gestire facilmente le chiamate AJAX in modo asincrono e sequenziale, consentendo di utilizzare i dati ricevuti dalla risposta per effettuare ulteriori operazioni come ad esempio effettuare una seconda chiamata AJAX.

Costruiamo ora una pagina che venga "popolata" usando proprio questo meccanismo della Promise:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Fetch API Example</title>
  </head>
  <body>
    <div id="todo"></div>
    <div id="user"></div>

    <script>
      fetch('https://jsonplaceholder.typicode.com/todos/1')
      .then(response => response.json())
      .then(todo => {
        // Popolamento della sezione Todo
        const todoDiv = document.getElementById('todo');
        const todoTitle = document.createElement('h1');
        todoTitle.textContent = todo.title;
        todoDiv.appendChild(todoTitle);
        // Ritorna il valore di todo per passarlo al prossimo then
        return todo;
      })
      .then(todo => {
        // Estrapolazione dell'ID dell'utente dal Todo
        const userId = todo.userId;
        return fetch(`https://jsonplaceholder.typicode.com/users/${userId}`);
      })
      .then(response => response.json())
      .then(user => {
        // Popolamento della sezione User
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ecco l'output:



Figure 1 *- Fetch API Promise*

Ecco una descrizione del codice:

- Si effettua una chiamata AJAX alla risorsa https://jsonplaceholder.typicode.com/todos/1 utilizzando il metodo fetch().
- Si converte la risposta in formato JSON utilizzando il metodo json() e si passa il risultato alla funzione di callback del primo then.
- Si popola la sezione "Todo" della pagina HTML con il titolo del ToDo ottenuto dalla risposta.
- Si utilizza il valore di todo ottenuto dal primo then per estrarre l'ID dell'utente e si effettua una seconda chiamata AJAX alla risorsa https://jsonplaceholder.typicode.com/users/{userId}
- Si converte la risposta in formato JSON utilizzando il metodo json() e si passa il risultato alla funzione di callback del terzo then
- Si popola la sezione "User" della pagina HTML con il nome dell'utente ottenuto dalla risposta.
- In caso di errore in una qualsiasi delle chiamate AJAX, viene gestito l'errore con il metodo .catch().

In sostanza, il codice effettua due chiamate AJAX in sequenza e utilizza i risultati ottenuti per popolare due sezioni della pagina HTML. La seconda chiamata AJAX dipende dal valore ottenuto nella prima chiamata; quindi, viene gestita all'interno della catena delle Promise. In caso di errore, viene gestita una eccezione con il metodo .catch().

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. CORS

Il CORS è un meccanismo di sicurezza utilizzato dai browser web per prevenire attacchi cross-site scripting (XSS) e per limitare l'accesso alle risorse di un sito web da parte di altre origini.

In particolare, il CORS prevede che un sito web possa accedere alle risorse di un altro sito web solo se quest'ultimo esplicitamente consente l'accesso attraverso l'utilizzo di apposite header HTTP. Ciò significa che se si tenta di effettuare una richiesta AJAX a un sito web che non ha abilitato il CORS, il browser impedirà la richiesta e restituirà un errore.

Per risolvere il problema del CORS, è possibile adottare diverse soluzioni.

3.1 Access-Control-Allow-Origin

Per consentire l'accesso alle risorse di un sito web attraverso il CORS, è necessario configurare il server di destinazione per inviare appositi header HTTP nelle risposte alle richieste AJAX provenienti da altri domini.

In particolare, il server deve inviare l'header **'Access-Control-Allow-Origin**' nella risposta HTTP, indicando gli origini (domini) dai quali accettare le richieste. Ad esempio, per consentire l'accesso alle risorse del sito web 'example.com' da parte di una pagina web ospitata su 'mywebsite.com', il server di 'example.com' deve inviare l'header 'Access-Control-Allow-Origin: mywebsite.com' nella risposta HTTP.

È possibile utilizzare anche altri header, come 'Access-Control-Allow-Methods' e 'Access-Control-Allow-Headers', per specificare i metodi HTTP e i tipi di dati accettati dalle richieste AJAX provenienti da altri domini.

In pratica, la configurazione del CORS dipende dalla piattaforma e dal framework utilizzati per sviluppare il server di destinazione. Ad esempio, in caso di sviluppo di un server web basato su Node.js, è possibile utilizzare il middleware 'cors' per semplificare la configurazione del CORS.

Se il server di destinazione ha configurato correttamente il CORS, il client può effettuare richieste AJAX senza alcun tipo di restrizione tecnica.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3.2 Proxy Server

Un'altra opzione è quella di utilizzare un proxy server per effettuare le richieste AJAX in modo da evitare le restrizioni del CORS. In questo caso, la richiesta AJAX viene inviata al proxy server, che a sua volta invia la richiesta al server di destinazione e restituisce la risposta al browser.

Per utilizzare un proxy server per effettuare le richieste AJAX e evitare le restrizioni del CORS, è necessario che il proxy server sia stato abilitato e configurato correttamente.

Un proxy server è un server intermedio che agisce come un ponte tra il client (cioè il browser) e il server di destinazione. Quando il client effettua una richiesta AJAX, invece di inviarla direttamente al server di destinazione, la richiesta viene inviata al proxy server. Quest'ultimo si occupa di inoltrare la richiesta al server di destinazione, ottenere la risposta e restituirla al client.

In questo modo, il client non effettua direttamente la richiesta al server di destinazione e il server non è soggetto alle restrizioni del CORS. Il proxy server agisce come intermediario e consente di superare le limitazioni imposte dal CORS.

Tuttavia, l'utilizzo di un proxy server per effettuare le richieste AJAX può comportare alcune complicazioni, come ad esempio la configurazione del server proxy e la gestione delle eventuali autenticazioni richieste dal server di destinazione.

In generale, l'utilizzo di un proxy server per evitare le restrizioni del CORS può essere una soluzione utile in alcuni casi, ma richiede una maggiore attenzione alla configurazione del server proxy e alla gestione delle richieste AJAX.

3.3 JSONP

Il JSONP (JSON with Padding) è una tecnica utilizzata per effettuare richieste AJAX a un server di destinazione che non supporta il CORS.

In pratica, quando il server di destinazione non supporta il CORS, il browser impedisce l'accesso alle risorse attraverso le richieste AJAX standard. In questo caso, è possibile utilizzare il JSONP per ricevere i dati JSON dal server di destinazione. Il JSONP prevede l'utilizzo di una callback per ricevere i dati JSON dal server di destinazione. In pratica, la richiesta AJAX non viene inviata attraverso XHR o Fetch API, ma viene effettuata tramite un tag script HTML.

Quando viene effettuata la richiesta AJAX tramite JSONP, il browser crea un tag script HTML con l'URL della risorsa da richiedere. Nell'URL viene specificata una callback, ovvero il nome di una funzione JavaScript che verrà chiamata quando il server di destinazione restituirà i dati JSON.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il server di destinazione restituisce quindi i dati JSON incapsulati nella funzione specificata nella callback. Quando il browser riceve la risposta, esegue la funzione JavaScript specificata nella callback, passando i dati JSON come parametro.

In questo modo, il JSONP consente di effettuare richieste AJAX a un server di destinazione che non supporta il CORS. Tuttavia, il JSONP può comportare alcune complicazioni, come ad esempio la necessità di specificare il nome della callback nel codice JavaScript e la gestione degli eventuali errori che possono verificarsi durante la richiesta.

Inoltre, il JSONP presenta alcune limitazioni, come ad esempio la mancanza di supporto per le richieste POST e la necessità di utilizzare un formato di dati particolare (ovvero il JSON incapsulato in una funzione JavaScript).

Ecco di seguito un esempio riassuntivo di chiamata AJAX tramite jquery usando jsonp e senza:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Esempio JSONP con jQuery</title>
  </head>
  <body>
    <h1>Esempio JSONP con jQuery</h1>
    <button id="ajax-button-jsonp">Effettua richiesta</button>
    <div id="resultJsonp"></div>

    <h1>Esempio JSON con jQuery</h1>
    <button id="ajax-button-json">Effettua richiesta</button>
    <div id="resultJson"></div>


    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

    <script>
      function handleResponseJsonp(response) {
        // Funzione chiamata da JSONPlaceholder con i dati JSON come parametro
        console.log(response);

        // Mostra il risultato nella pagina HTML
        var resultDiv = document.getElementById("resultJsonp");
        resultDiv.innerHTML="<h2>Risultato:</h2><pre>"+  JSON.stringify(response,  null,
2) + "</pre>";
      }

      // Effettua la richiesta AJAX tramite JSONP al click del pulsante
      $('#ajax-button-jsonp').click(function() {
        $.ajax({
          url: 'https://jsonplaceholder.typicode.com/posts/1',
          dataType: 'jsonp',
          jsonpCallback: 'handleResponseJsonp'
        });
      });
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
    </script>

    <script>
      // Effettua la richiesta AJAX al click del pulsante
      $('#ajax-button-json').click(function() {
        $.ajax({
          url: 'https://jsonplaceholder.typicode.com/posts/1',
          dataType: 'json',
          success: function(response) {
            // Funzione chiamata al successo della richiesta AJAX
            console.log(response);

            // Mostra il risultato nella pagina HTML
            var resultDiv = document.getElementById("resultJson");
            resultDiv.innerHTML = "<h2>Risultato:</h2><pre>" + JSON.stringify(response,
null, 2) + "</pre>";
          },
          error: function(xhr, status, error) {
            // Funzione chiamata in caso di errore della richiesta AJAX
            console.log("Errore: " + error);

            // Mostra l'errore nella pagina HTML
            var resultDiv = document.getElementById("resultJson");
            resultDiv.innerHTML = "<h2>Errore:</h2><p>" + error + "</p>";
          }
        });
      });
    </script>

  </body>
</html>
```

Nella sezione relativa alla chiamata con **JSONP**, quando il server di JSONPlaceholder restituisce i dati JSON, la funzione 'handleResponseJsonp' viene chiamata dal browser, passando i dati JSON come parametro; tale funzione mostra il risultato nella pagina HTML, nel div con l'ID 'resultJsonp'.

Nella sezione relativa alla chiamata AJAX standard, viene definita la funzione success che verrà chiamata al successo della richiesta AJAX e la funzione error che verrà chiamata in caso di errore della richiesta AJAX. Viene quindi utilizzata la funzione click di jQuery per effettuare la richiesta AJAX al click del pulsante; quando l'utente clicca sul pulsante, viene effettuata la richiesta AJAX tramite il metodo $.ajax di jQuery, specificando l'URL della risorsa da richiedere e il tipo di dato (JSON). Quando il server di JSONPlaceholder restituisce i dati JSON, la funzione success viene chiamata dal browser, passando i dati JSON come parametro. La funzione success mostra il risultato nella pagina HTML, nel div con l'ID 'resultJson'. In caso di errore della richiesta AJAX, la funzione error viene chiamata dal browser, passando le informazioni sull'errore come parametro. La funzione error mostra l'errore nella pagina HTML, nel div con l'ID 'resultJson'.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In conclusione, per poter effettuare una chiamata AJAX ad un server che non supporta CORS è possibile usare JSONP tramite un tag script HTML, ma l'unico metodo consentito in questo caso è GET. Un server che supporta CORS, invece, può accettare richieste in qualsiasi metodo HTTP e in qualsiasi formato dati, inclusi sia JSON che JSONP.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010.
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR, 2005.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Sitografia

- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).